Automation in Construction 74 (2016) 28–38

Contents lists available at ScienceDirect

Automation in Construction

journal homepage: www.elsevier.com/locate/autcon

Robotic fabrication of freeform foam structures with quadrilateral and puzzle shaped panels

Marko Jovanović*, Mirko Raković*, Bojan Tepavčević, Branislav Borovac, Milutin Nikolić

University of Novi Sad, Faculty of Technical Sciences, Trg Dositeja Obradović 6, Novi Sad, Serbia

ARTICLE INFO

Article history:
Received 19 November 2015
Received in revised form 18 August 2016
Accepted 2 November 2016
Available online xxxx

Keywords:
Robotic fabrication
Freeform structures
Puzzle shaped panels
Hot-wire cutting
Shape tessellation

ABSTRACT

In this paper a novel methodology is proposed for digital workflow and fabrication of a freeform shell made from foam materials utilizing a robot hot-wire cutting tool. The major issues of using freeform foam structures pertain to building complex shape constructions in contemporary architecture and fabrication of structural, insulated panels for double-curved walls. In order to provide an efficient method of freeform surface panelization, two different tessellation approaches are presented - a quadrilateral shaped panel and a puzzle shaped panel. A puzzle shaped panel is chosen for fabrication purposes, in order to test the benefits that come with it, including friction fit connections, and assemblability. Digital workflow based on the graphical algorithm editor Grasshopper is used for the development of complex panel shapes derived by tessellation procedures. The proposed method is tested through fabrication of a full scale architectural prototype.

© 2016 Elsevier B.V. All rights reserved.

1. Introduction

With increased use of digital technologies in contemporary architecture, new challenges, regarding fabrication with the respect of material properties, emerged. Most materials used in architecture, such as wood, metal or glass, are produced in a form of flat sheet materials, mainly due to cost-efficiency. Making double-curved panels for architectural facades is often a time consuming and expensive task. Consequently, double-curved surfaces in architectural design are often approximated with a set of flat panels that can be manufactured at a reasonable cost. However, planar panels may have faceted appearances which may not satisfy the aesthetic criteria of the design due to reflections and shadows on the facade. Thus, from the aspect of visual appearance, curved panels might be more desirable than design based on flat panels. Optimal ruled surface fitting and rationalization of freeform surfaces to ruled surface strips can mitigate or completely remove the faceted appearance [1,2] and they are important issues for modeling in architecture. Recent advances in architectural geometry provide cost-efficient solutions for segmentation of freeform surfaces with single-curved panels [3–5]. This research focuses on fabricating freeform structures, composed of ruled surface patches. Provided that single curved surfaces are ruled surfaces with exclusively torsional generators, they can be joined to form a freeform surface of any kind.

Furthermore, recent advances in fabrication technology are directed toward automated production of double and single-curved panels [6,7]. However, construction methods generally rely on material properties. Digital design and fabrication workflow designed for one type of material is often not suitable for other solutions and materials. Emerging ideas and principles in architectural design based on interconnection between digital design, fabrication process and advanced material design are defined as Digital materiality [8] or Material Ecologies [9].

Generation of freeform foam structures has already been utilized by CNC milling machines and more recently with robot hotwire tools. However, integration of tessellation procedures into digital workflow that considers waste material minimization and assemblability has not been utilized yet. This paper presents a novel methodology and algorithmically guided workflow for designing and using robotic fabrication of freeform foam structures, based on different tessellation procedures. It considers material properties and fabrication workflow (digital materiality/material ecologies), in order to reduce scrap material and optimize the connection of elements in the assembly process.

Two tessellation procedures are explored in terms of geometry generation, a quadrilateral shaped panel and a puzzle shaped panel, emphasizing the benefits and contributions of their utilization. However, a puzzle shaped panel is chosen for fabrication purposes to test the benefits of friction fit connections, which it allows, assemblability

* Corresponding authors.
E-mail addresses: markojovanovic@uns.ac.rs (M. Jovanović), rakovicm@uns.ac.rs (M. Raković).

http://dx.doi.org/10.1016/j.autcon.2016.11.003
0926-5805/© 2016 Elsevier B.V. All rights reserved.

challenges and opportunities, alongside fabrication workflow and on site applicability. Although puzzle shape panels have been used in some design project [10,11], digital fabrication workflow presented here provides a solution that considers thick puzzle shaped elements that need to be cut by a 5 or 6 axis fabrication tool.

Apart from fabrication-aware design, major advantage of the proposed workflow is the utilization of a single software environment (Grasshopper3D/Rhino by David Rutten at Robert McNeel & Associates) for design-based processes (form generation, rationalization and tessellation) and fabrication processes (generating paths and code for the robotic fabrication). The methodology presented may be utilized for fabrication of structural insulated panels for double-curved walls as well as for concrete cast molds. Also, this similar approach may be used with wire diamond cutting mounted on a robotic arm for fabrication of stone or concrete blocks of unique shape [12,13]. The methodology is introduced in the following steps:

- freeform surface rationalization with ruled surfaces,
- tessellation algorithms,
- numeration and packing the tessellated elements into blocks,
- generation of cutting paths,
- generation of a code for robot controller
- robotic fabrication with ABB IRB 140 industrial robot and hot-wire cutting tool and
- wall assembly.

The rest of this paper is organized as follows. After related work in Section 2 and a freeform surface rationalization in Section 3, two different tessellation algorithms are presented in Section 4. The first algorithm, the box-like tessellation, deals with the problem of reducing scrap material, due to its shape and size of tessellated elements. The second algorithm, the puzzle-like tessellation, pertains to the generation of elements, optimized for efficient connection, which do not require additional adhesive agents. After tessellating, in Section 5 the numeration and packing of the tessellated elements into a Styrofoam box and the generation of cutting paths is presented. Based on generated cutting paths, in Section 6 the procedure for generating the code for robot controller is described. Once the code for robot controller is generated, it enables the robot to perform the necessary movement for cutting out the elements. In Section 7, the last two steps are presented, the verification of the design system workflow through the process of foam elements fabrication and their assembly into an architectural prototype. Finally the paper is concluded in Section 8.

2. Related work

Free form elements are generally produced by applying custom designed molds milled by CNC milling machines or wire cutting [14,15]. CNC foam milling is often time consuming and costly but it is still the most efficient solution for some cases [16,17], such as the Spencer Dock Bridge in Dublin by Future Systems [18]. On the other side, wire cutting is generally more efficient, but constrained with design possibilities regarding the shape of the structures, material selection[1] , compared to milling techniques.

Recently, advantages in industrial robotic arms utilization in digital fabrication processes are recognized, influencing the increased replacement of common digital fabrication technologies [19]. Robotic arms are beginning to replace numerous fabrication techniques including 3D printing processes [20], brick laying [21–23], tile placement [24], filament winding [25], metal folding [26], milling [27], slip forming [28], bending double-curved ribs [29] and wire cutting [30].

Robotic fabrication with foam materials is an important topic, since it has been used both as a casting technique and as an element in structural, insulated panels.

Design innovation of freeform concrete structures such as Tailor Crete by Gramazio & Kohler [31] or fabric formwork research by Mark West at C.A.S.T [32] re-raised the question of design possibilities in building industry with concrete. At the same time, demands toward energy efficient design questioned cost-efficient solutions for such structures' insulation systems. The same question may be raised for the advanced design of metal facades with curved folded panels that can be seen in research projects of Robofold or Gramazio & Kohler. For all mentioned examples styrofoam might be applied as cost-efficient insulation system solution.

Robotic wire cutting of foam structures has already been used in the form-finding design process [33] and in customizing components for larger structures [34] as a possible answer to the new design demands. However, prior freeform surface rationalization and tessellation is necessary to begin cost-efficient fabrication in the first place. Utilizing software tools for freeform surface rationalization [35–37] including D.Loft enables generating appropriate elements with controlled tolerances providing solutions for a specific manufacturing process in architecture, such as sheet metal and wood veneer bending [2]. Also, such tools can be used in the preprocessing step of the tessellation algorithm or as a separate step altogether, when dealing with more complex shapes and surfaces. In this research optimized tessellation procedures are integrated in the design and fabrication process of freeform structure's workflow.

Tessellation algorithms are extensively used for various architectural design strategies, but there are only a few examples of utilizing puzzle shape tessellation procedures. The most interesting property of puzzle shaped tessellation is the self-interlocking connection. Such property is used in Axel Kilian's design experiments for connecting double curved surfaces of various materials into a single surface [39]. Self-interlocking puzzle connection is also used in designing self-supporting structure Towering Jigsaw Puzzle Curtain Made of Stone project, where a water jet cut the marble stone into pieces for later assembly [11]. However in the both mentioned projects self-interlocking connection exists only in one direction and puzzle like elements are flat and have inappreciable thickness comparing to its size. In this research freeform surface rationalization and optimized tessellation procedures are integrated in the design and fabrication process of freeform structure's workflow.

3. Freeform surface rationalization with ruled surface

Digital workflow starts with the generation of a freeform surface. The next step is the generation of piecewise ruled surfaces that approximate the given freeform surface. For this research, software package Rhinoceros is used with its plug-in Grasshopper. Kangaroo add-on for Grasshopper plug-in can be used for planarization of parallel cross sections of a surface making it ruled. Mentioned tools as well as custom script components might be used for greater control of kink angles between neighboring ruled surface.

In this research we focus on providing one ruled surface element from the freeform surface to enable testing with physical prototypes. A freeform surface is generated from a network of intersecting curves (Fig. 1a). Then, two isoparametric curves are extracted in both directions, dividing the surface into 9 pieces (Fig. 1b). Isocurves in one direction are converted into polylines with the vertices placed on intersections with the other set of isocurves. The new network set of intersecting curves are used for the generation of a piecewise

[1] Wire-cutting tools can be hotwire cutting and abrasive wire cutting. Hot-wire cutting tool is most often used to cut polystyrene foam and similar materials like Styrofoam. Abrasive wire cutting, such as diamond-wire cutting, allows for cutting concrete elements, stone etc.

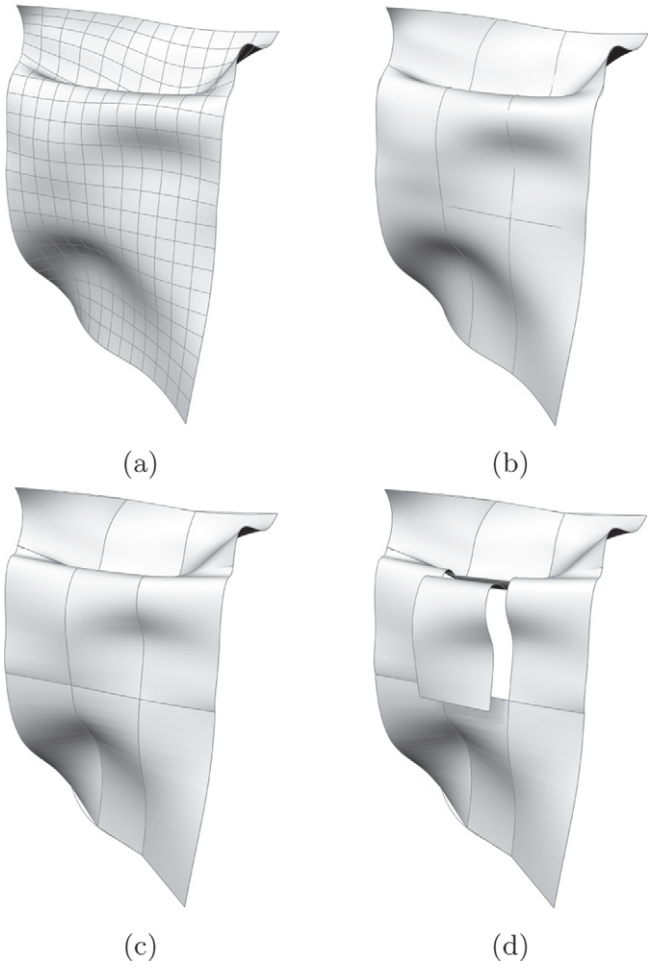


Fig. 1. Freeform surface rationalization process and selection of a desired ruled surface result for fabrication.

ruled surfaces that approximates the given surface (Fig. 1c). Lastly, one ruled surface section is used in digital workflow design and fabrication process (Fig. 1d).

4. Tessellation algorithm

The digital workflow continues with ruled surface referencing and tessellation into smaller elements for fabrication. The tessellation procedure depends on two parameters, the shape and the size of the final elements. In this research two different shapes in tessellation procedure are used: box-like elements and puzzle-like elements (Fig. 2). The first one implements generating elements with straight edges that should be connected via silicone, coupling elements or adhesive agents. The second approach implements generating puzzle like elements, that do not require additional coupling elements or adhesive agents, but use a bit more material during the fabrication process due to the irregularity of its shape. Both approaches will be described.

The design code tessellates the surface into elements which are to be fabricated. The code automatically generates data pertaining to their fabrication. The data consists of a set of points and their corresponding vectors which determine the position and orientation of the hot-wire cutter tool in space and the cutting path in general.

The size of the elements to be fabricated is dependent on the dimensions of the cutting boxes, which are derived from the initial

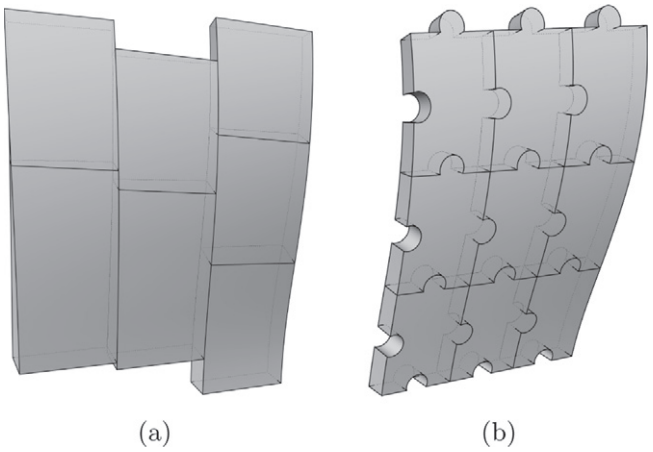


Fig. 2. Two examples of tessellating a single curved surface: a) box-like tessellation; b) puzzle-like tessellation.

Styrofoam block, as well as on the hotwire cutting tool size[2]. Depending on the value of those parameters, aiming to maximize the use of the material available, and reduce the scrap material, a condition is introduced, which determines what the maximum width of the cutting box is:

$$W_t \geq W_{ib} \Rightarrow W_b = W_{ib} \qquad (1)$$
$$W_t < W_{ib} \Rightarrow W_b = \frac{W_{ib}}{round\left(\frac{W_{ib}}{W_t}\right)} - T$$

In Eq. (1), W_t is the width of the hotwire tool, T is the thickness of the hotwire, W_{ib} is the width of the initial Styrofoam block, and W_b is the width of the cutting box used in the final robotic fabrication process (Fig. 3).

Once W_b is determined, two approaches to tessellating the curved surface and generating box-like and puzzle-like elements are analyzed.

4.1. Box-like tessellation

Box-like tessellation generates quad faced parts. The procedure starts by determining the width of the elements based on the surface generatrix length and cutting box width W_b as: $W_{Box} = W_s/round(W_s/W_b)$, where W_s is the generatrix length of the ruled surface. Next, the surface is divided into stripes, with the designated cutting box width W_{Box}. Afterwards, the stripes are tessellated into elements, which can later be fabricated from the cutting box. For determining the height of the elements $H_{i,j}$ (where i is the index of stripe, and j is index of element in current stripe) the following process is done.

The procedure starts by selecting an i-th stripe, where $i \in [1, n]$, and n is the number of stripes (Fig. 4a). Then, from one end of the directrix, length measuring the initial height of the element $H_{i,j}$, is set to be equal to the height of the cutting box. Further, a desired thickness is applied to the element and it is evaluated whether the element fits into the dimensions of the cutting box. If the element does not fit, the $H_{i,j}$ is iteratively decreased by value h, until it fits into the size of the cutting box. The lesser the value h is, the more precise the calculation is, the less material is wasted, but the tessellation process lasts longer. After the height of the element is determined,

[2] The properties and adjustments of the hot-wire cutting tool itself will be the topic of Section 7 and will be discussed later on.

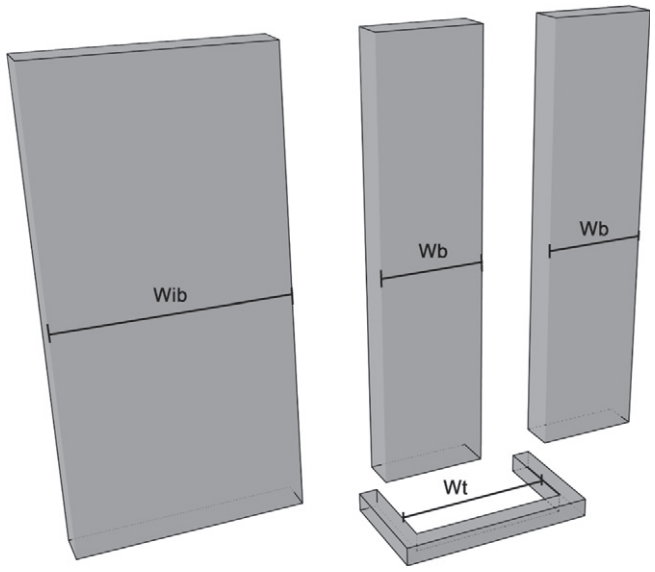


Fig. 3. An example of an initial Styrofoam block on the left and two cutting boxes with the cutting tool on the right.

it is culled from the initial stripe. From the remaining part of *i*-th stripe, other elements are iteratively determined the same way, until the other end of diretrix is reached (see Fig. 4b). The procedure is repeated for each stripe, until all the elements are generated, i.e. until the complete surface is tessellated. It is important to notice that elements are smaller as the minimal radius of surface curvature at that part of the stripe decreases. The radius of curvature and the corresponding box-like tessellation of the surface are illustrated in Fig. 4c.

4.2. Puzzle-like tessellation

The main advantage of the puzzle-like tessellation approach is the simple connection between elements in the assembly process based on self-interlocking [38,39]. The puzzle curves are defined on a planar surface and then mapped onto a desired ruled surface. To achieve this, it is necessary to measure each edge of the initial surface. Afterwards, the dimensions of the two longest adjacent edges, the diretrix and the generatrix are taken and a planar surface is generated, matching those dimensions.

For defining puzzle curves on a planar surface, the surface is tessellated into quadrilaterals (Fig. 5a). The width of quadrilaterals W_q is calculated as $W_q = W_s/round(W_s/W_b) - h_a$, where h_a is the height of the convex arc (see Fig. 5c). The height of the quadrilaterals is determined in reference to the height of the cutting box and is user defined.

Therefore, the height of the convex arc determines the reduction in element width. When the quadrilaterals are generated, the midpoints of each edge are moved by a user defined value toward or away from the center of the quadrilaterals, depending on the desired arc type - convex or concave (Fig. 5b). These points serve as the center for the arcs (Fig. 5c). The trimmed edges and arcs are then joined into a single closed curve respectively (Fig. 5d), and mapped to the ruled surface (Fig. 5e) using a prewritten component in the Grasshopper visual programming language environment. The component maps the curves' UV coordinates from a planar to a ruled surface of the same size. Due to the surface curvature, arising curve distortions are imminent but not relevant since they don't affect the fabrication workflow. Next, the desired thickness of the wall is added and it is checked if all the elements fit into the cutting box or what their thickness should be if they are to be fabricated from the cutting box. It is in this manner that the elements for the fabrication process are generated. Both tessellation procedures as part of the workflow are best suited for case specific conditions that arise during robotic hotwire fabrication and requirements. Afterwards, the design code numbers and packs the elements into cutting boxes, producing minimum waste material, and generating the cutting paths necessary for the robotic fabrication process.

5. The cutting path

After the elements have been generated through the tessellation process, they need to be numbered and packed, as a set, in the cutting boxes. The numeration process is carried out in reference to the element's bounding box height in ascending order. Height is taken as a key factor, since the width and thickness of the elements are set to fit those of the cutting box. The numeration process is also important for the assembly process later on.

The algorithm sorts the elements, based on bounding box height, to fit as many elements in a single cutting box as possible. This is important for the generation of the cutting path later on and for reducing the scrap material during fabrication. The average number of elements in a single cutting box is determined as the rounded result of the height of the cutting box divided by the average height of the entire list of elements.

For the fabrication of a full scale prototype, the puzzle-like tessellation algorithm is chosen. Further on in the paper puzzle-like elements are considered as well. In the case of puzzle-like elements and for the surface used in this research, the height of the elements is such that from one cutting box up to three puzzle elements can be cut out. To minimize the scrap material, the algorithm for selecting the set of three elements is introduced. The algorithm first finds the first and second highest available elements, and searches for the third element that can fit together with the first two into the height of the cutting box. If the algorithm cannot find the third element, the second element is replaced with the shorter one and again the third element is searched. This procedure is repeated until all the elements are grouped into a set of three elements. An example of one set of three elements is shown in Fig. 6a.

In order to generate a set of instructions that the robot can follow and fabricate the packed elements from the cutting box, the cutting paths need to be determined. The cutting path consists of a myriad of points, where each point is determined by its position and orientation. In addition, the cutting path incorporates supporting points for approaching the cutting box and for safe movement of the hot-wire cutter away from the cutting box. Before the path generation process can begin, the cutting box needs to be positioned with respect to the robot's reference coordinate frame which ensures that the cutting path is within the reachability of the robot's cutting tool.

There are two cutting paths for each cutting box (Fig. 7). The first cutting path, defining the front and back sides of the elements, cuts the part of the initial ruled surface, a patch, pertaining to the element being fabricated (Fig. 7a). The second cutting path pertains to the outline of the puzzle-like element shape itself (Fig. 7b). The outlines of the ruled surface patches are referenced and then evaluated according to their respective extremes in the vertical plane. The lowest and the highest point are thus defined, so that the entirety of the element is cut during fabrication, including the concave arc shapes.

According to the lowest and highest points previously defined, geodesic lines are then generated on the surface patches. The lines are divided into a number of segments, and the end point of each is taken. In the next step, the surface is evaluated at the given points and the normal vector and tangent plane is determined accordingly. The tangent plane is defined by two vectors, out of which one coincides with the generatrix direction, necessary to define the hot-wire alignment during the cutting process. That vector, denoted as the

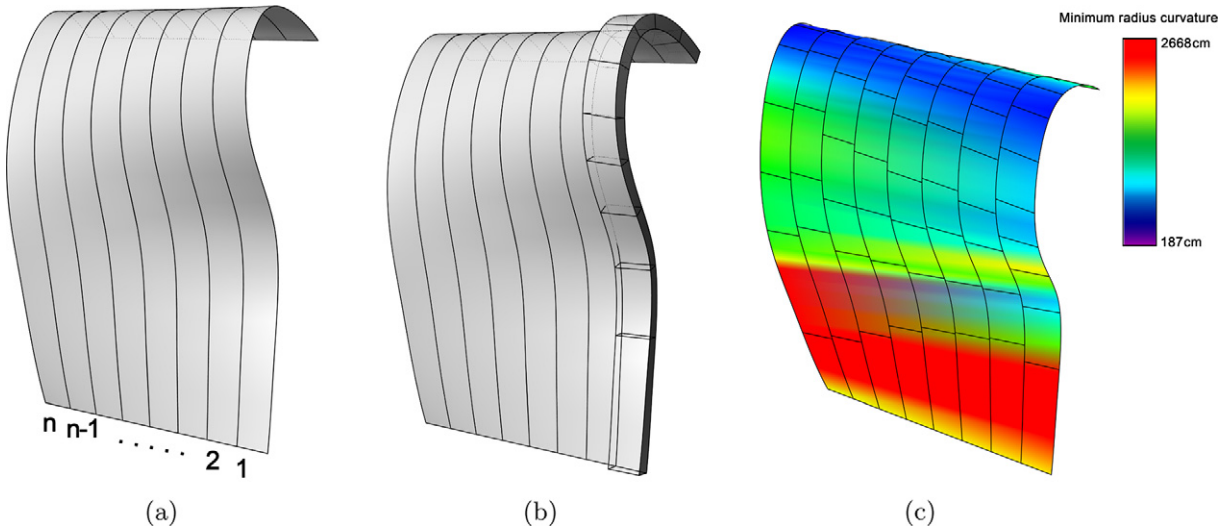


Fig. 4. Box-like tessellation of the ruled surface: a) Division of surface into stripes; b) Determined box-like elements for one stripe; c) Dependence of the minimum radius curvature and the size of the surface quadrilaterals.

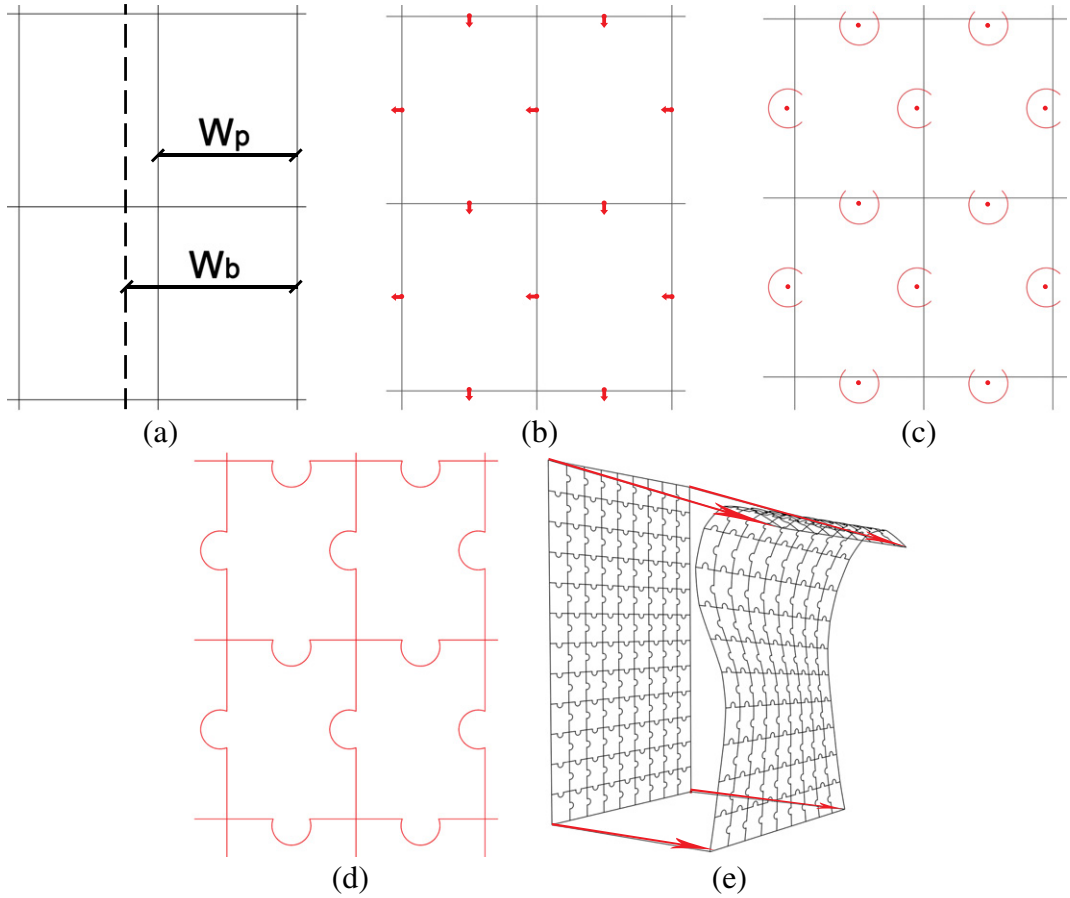


Fig. 5. The process of generating puzzle like elements - a) generating planar quadrilaterals; b) moving the edge midpoints; c) generating arcs using points as centers; d) trimming excess lines; e) mapping the curves onto a surface.

vector $\vec{v}$ is taken as a starting point and a unit vector $[0,0,1]^T$, in reference to the global coordinate system, is introduced. The global coordinate system is the same one where the cutting box is placed for later fabrication, meaning that the element height is aligned to the said vector. Cross product between $\vec{v}$ and $[0,0,1]^T$ generates the vector $\vec{u}$. The last step is to calculate the cross product of vectors $\vec{v}$ and $\vec{u}$, which generates the vector $\vec{w}$ (see Fig. 6b). In such a manner, all the data for the first cutting path are generated. This includes the

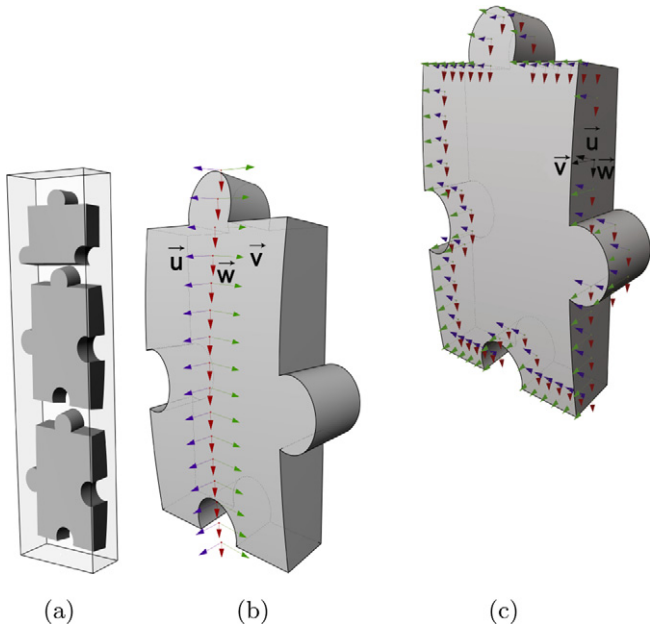


Fig. 6. Cutting path: a) Position and orientation of puzzle like elements prepared for the path generation process inside a cutting box; b) Orientation vectors in points on the first cutting path, i.e. for cutting the front ruled surface patch; c) Puzzle outline cutting path and the vectors representing the direction of the hot wire cutter tool.

three coordinates in the reference coordinate frame and the three vectors, referencing the point's orientation. The hot-wire is aligned to the vector $\vec{v}$, while the vector $\vec{u}$ generates an orientation plane for the hot-wire tool, important for avoiding collisions. The third vector, i.e. vector $\vec{w}$ is necessary for forming the right-handed coordinate system in order to determine target positions of the hot-wire tool during the cutting process. Since there are three elements vertically stacked within a single cutting box, the points between two neighboring elements are connected with a line generating a cutting path for both front and back sides respectively (Fig. 7a).

For generating the second cutting path the outline of the surface patch is extracted and exploded into curves. The closed curve is then divided into a desired number of segments, whose end points are referenced, including the start and end point for each curve. This guarantees the lack of a curve corner filleting and the absence of an imprecise cutting procedure. Afterwards, the points are offset from their initial position as to accommodate for the thickness of the wire. An interpolated closed curve referencing these points represents the second cutting path. The orientation for each point needs to be determined as well, in order to have all the necessary data. Three vectors in the respective points are determined in the same manner as for the points which constitutes the first path (Fig. 6c).

In such a manner, all the position and orientation data, which represent the necessary targets for the robot, are procured. In order to guarantee an accurate and precise fabrication procedure, the targets need to be aligned and sorted along their respective cutting paths. Only when the targets are sorted, the extraction and calibration of the data for the robot controller part of the code can begin.

6. The code for robot controller

For enabling the robot to perform the necessary movement for cutting the elements, two programs have been developed: the program for robot controller written in RAPID[3] , and the Grasshopper C# script for generating add-on code for the RAPID program.

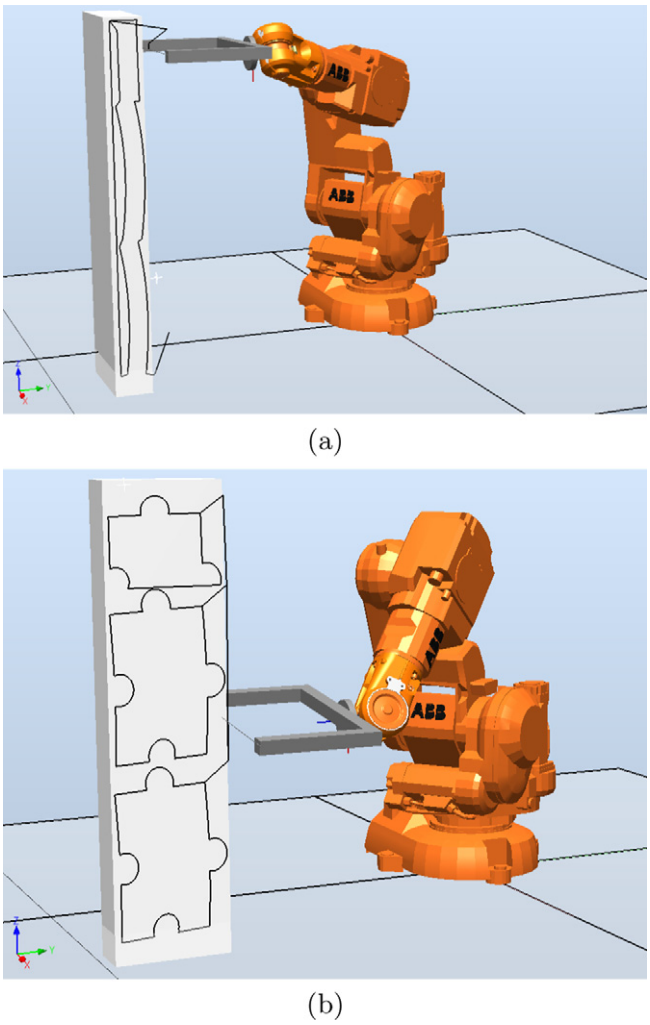


Fig. 7. Position of the foam block a) for cutting the front and back part of puzzles and b) for final cutting out the puzzles.

6.1. The RAPID program

For the development of the RAPID program and simulation of cutting path execution, the RobotStudio[4] software is used. Offline programming and simulation in RobotStudio offers a fast and reliable procedure for testing the reachability of path execution and for collision detection. Since the cutting tool is wide, there is a possibility for collision between the tool and the robot structure itself. To avoid this, and to maximize the robot's working area, the points on the path are selected (named targets) as such that the cutting tool is aligned as close as possible to the horizontal plane. If this is fulfilled, the tool will be horizontal as much as possible, while traveling between targets.

The RAPID program is organized in two modules: the module with the main program and the module with the definition of targets used for executing the cutting paths. The algorithm of the main

[3] RAPID is a programming language for controlling the ABB industrial robots.
[4] RobotStudio is the simulation and offline programming software for ABB industrial robots.

program (shown below) is organized in such a way that it is independent from the cutting paths, i.e. of the geometry of the puzzle-like elements that has to be cut out.

Algorithm 1. RAPID program.

```
Data: Position and orientation of all targets;
Result: Path for hot-wire cutter initialization;
initialization;
while true do
    wire_cutter_initial_position();
    wait_for_confirmation_1();
    safe_exit_path_1();
    wait_for_confirmation_2();
    puzzle_cutting_path();
    safe_exit_path_2();
    wait_for_ confirmation_3();
end
```

According to Algorithm 1, at the beginning, the robot will position its Tool Center Point (TCP) in the initial position (Fig. 8). Then the operator will receive the message on the robot controller touch screen that the TCP is positioned correctly and the question if the Styrofoam block is ready for cutting the front and back side of the elements.

After confirmation, the path for cutting the front and back sides of the elements, starts. When the path is finished, the robot will move the hot-wire out of the cutting box. The second message appears on the touch screen, asking if the position for the execution of the second path is correct. At this point the operator is asked to reposition the foam block for cutting the final elements. When the path for cutting the elements is executed the robot moves the hot-wire to a safe position and informs the operator that the cutting process is finished and waits for the next foam block to be placed.

Fig. 7 shows the position of the foam block for cutting the front and back sides and for the final cutting of the parts, where the black line represents the path of the TCP, i.e. the hot-wire path. The RAPID program developed according to Algorithm 1, represents a base for cutting all the elements off the wall.

Since each element of the wall has a unique shape, every time new elements are produced, the targets on a cutting path have to be updated. The targets are stored in second RAPID modules, together with the information about the number of targets on each path, and the array of targets that constitute the corresponding paths.

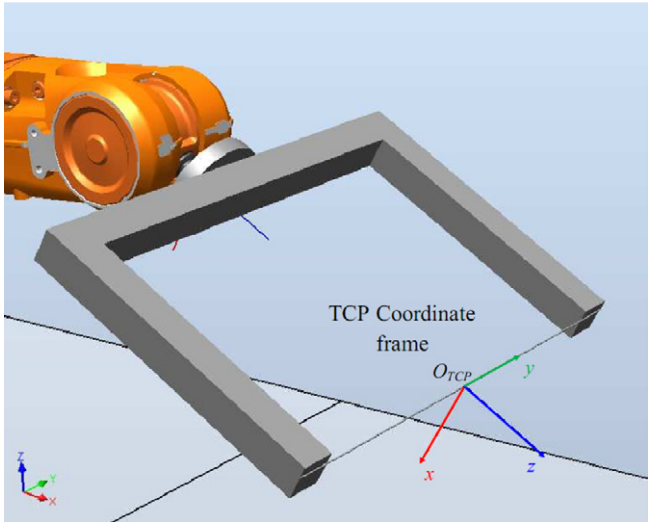


Fig. 8. The position and orientation of Tool Center Point coordinate frame for hot wire cutting tool.

6.2. The C# script

For automated transfer between the cutting path generated in the Grasshopper (Section 5) and the RAPID program (Sub-section 6.1), a Grasshopper C# script is written. The purpose of C# script is to generate a RAPID module, syntactically and semantically correct with respect to RAPID language, which incorporates the information about targets and paths for Algorithm 1. The input data used for the script consists of a set of points and the vectors which determine the position and orientation of the hot-wire cutter tool in space.

The position and orientation of the new targets are determined from the points and the associated vector of each element shown in Fig. 9. The reference coordinate frame is defined from one of four top corner points of the bounding box with the orientation aligned to a global coordinate frame. The position and orientation of the reference coordinate frame is denoted with $\mathbf{p}_{ref}$ and $\mathbf{o}_{ref}$.

The data structure for storing the information about the target in RAPID language is composed of the following fields:

- the position vector **p** (x, y and z) of the TCP,
- the orientation vector **o** of the TCP in the form of a quaternion (q_{xt}, q_{yt}, q_{zt} and q_{wt}),
- the axis configuration of the robot and the position of the external axes if it exists [40].

For each robot target, the position is calculated from $\mathbf{p}_t^i = \mathbf{p}_{glob}^{i-th} - \mathbf{p}_{ref}$, whereas the orientation $\mathbf{o}_t^i$ is calculated in the quaternion form $\left(q_{xt}^i, q_{yt}^i, q_{zt}^i, q_{wt}^i\right)$ by the following procedure. First the unit vector $\vec{t}_i$, $\vec{t}_j$ and $\vec{t}_k$ are calculated as:

$$\vec{t}_j = \vec{t}_{glob}^{\,i}, \vec{t}_i = \frac{\vec{k}_{ref} \times \vec{t}_{glob}^{\,i}}{\left|\vec{k}_{ref} \times \vec{t}_{glob}^{\,i}\right|}, \vec{t}_k = \vec{t}_i \times \vec{t}_j$$

which gives us the rotation matrix T:

$$T = \begin{bmatrix} t_{11} & t_{12} & t_{13} \\ t_{21} & t_{22} & t_{23} \\ t_{31} & t_{32} & t_{33} \end{bmatrix} = \left[\, \vec{t}_i \;\; \vec{t}_j \;\; \vec{t}_k \,\right]$$

Afterwards, the quaternions are calculated by utilizing the algorithm given in Appendix A. The axis configuration of the robot

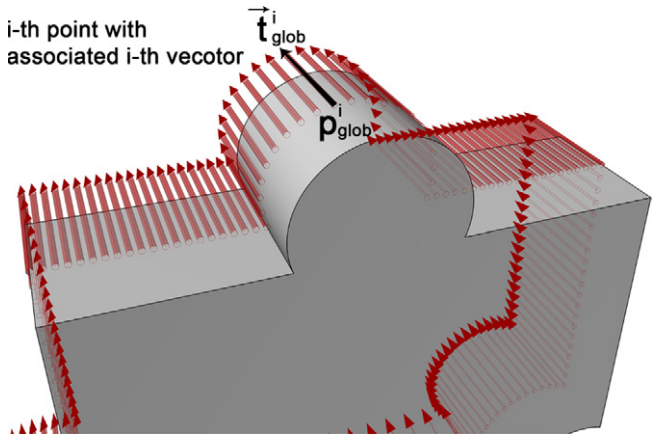


Fig. 9. Detail of one puzzle element with the associated vectors (red arrows) used for calculating the targets for RAPID program. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

and external axes are set to default values and do not change. After all the parts of the target data structure are known, the calculated values are merged into a code that is syntactically and semantically correct with respect to RAPID language. When the code for each of the targets is generated, they are assembled into an array of targets together with a header and footer to represent the second RAPID module ready to be imported into a RAPID program.

7. Realization of curved puzzle wall

The aforementioned described methodology is implemented into the realization of a full scale curved Styrofoam wall: 1.7 m wide, 5 cm thick and 2.1 m high, with the curved edges, i.e. directrices, having 3.2 m in length. The initial Styrofoam block is 60 cm wide, 125 cm high and 10 cm thick. An initial surface model is generated and tessellated into 108 puzzles-like elements in a 9×12 grid, according to the procedure described in the Sub-section 4.2 (Fig. 10). The elements are numbered in an ascending order - 0 being the shortest and 107 being the highest to enable sorting them into the sets of three puzzle elements in a single cutting box as explained in Section 5.

The first step in the fabrication process is cutting the initial Styrofoam block into the cutting boxes, based on the tool size (see Fig. 3). The cutting width of the tool is 40 cm and the thickness of the hot-wire is 2 mm. According to Eq. (1), the width of the cutting box is determined to be at around 29 cm, meaning that all the Styrofoam blocks used during fabrication are cut into two boxes having 29 cm width, 125 cm height and 10 cm thickness.

Once the code for the robot controller is generated, the production of the curved puzzle elements can begin. The cutting box needs to be oriented and positioned in the robot's workspace for the cutting process. Once the code for the robot controller is initiated, the robot positions the tool in the first point of the first cutting path. The cutting box is placed in front of it and the cutting process is initiated (Fig. 11), which is confirmed through the touch screen.

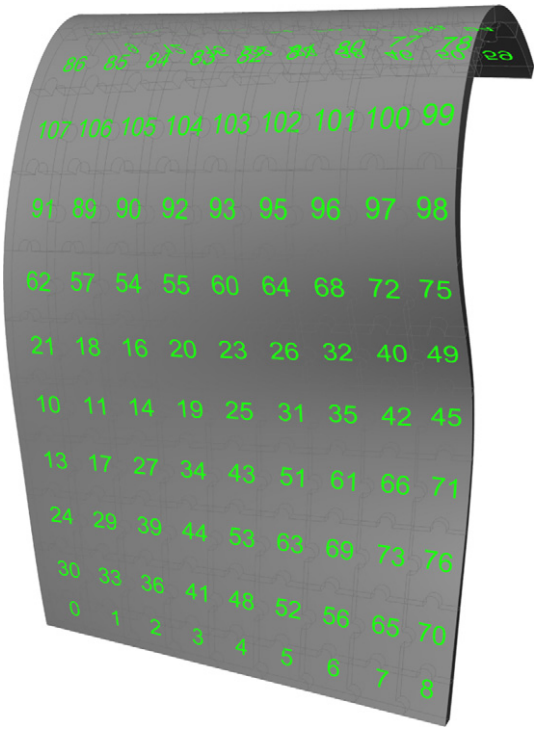


Fig. 10. A tessellated surface model with numbered puzzle elements.

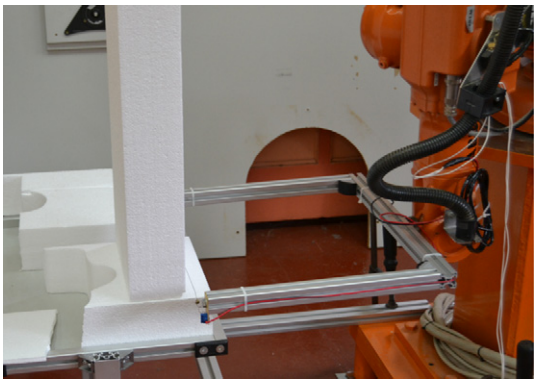


Fig. 11. The position of the cutting tool at the start of the first cutting path.

After the first cutting path is finished, the hot-wire cutter positions itself at the start of the second cutting path. Accordingly, the cutting box needs to be rotated into place, by aligning the side of the cutting box to the hot-wire cutter, i.e. by rotating the cutting box 90 ° clockwise. Then the second cutting path is initiated, generating the elements inside it (Fig. 12).

The second cutting path starts from the top puzzle-like elements and moves downwards. In order to reduce error during movement, the cutting path for each element runs clockwise. In such a manner, the top part of the element is being cut out at the end. This way the element will not move due to gravity, until it is cut out completely. After three puzzles are cut out from one cutting box, they are numbered in reference to their position in the curved wall (Fig. 13).

Once all the puzzle elements are cut out, the assembly process follows. Since this method relies on elements fitting together, the puzzles are integrated one into another forming a solid bond.



Fig. 12. The process of cutting out the puzzle elements during the second cutting path.

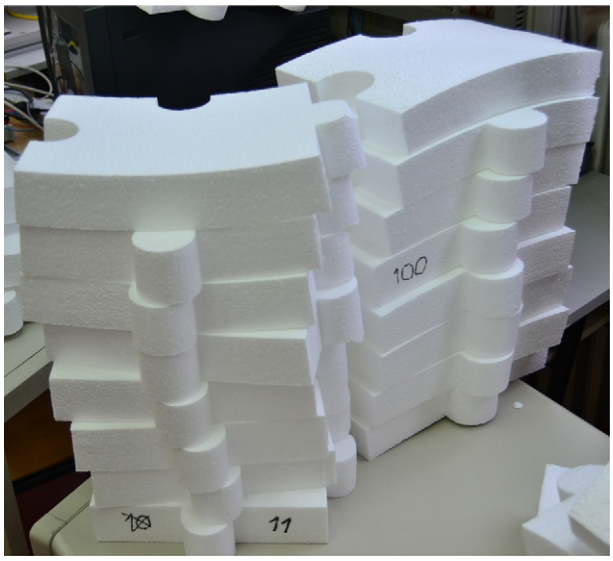


Fig. 13. A set of cut out puzzle elements numbered and ready for assembly.



Fig. 14. Three parts combined into one structure with pending insertion of planar puzzle elements in the missing spots.

The assemblability problem of elements that generate the freeform surfaces, is the insertion direction. The initially generated surface has changeable curvature throughout it (Fig. 4c). That is why a single puzzle piece is better fitted together when lesser surrounding pieces are around. Fitting a single row of elements is not difficult, since insertion direction only has to honor a single puzzle arc connection. Generating more finished rows and putting them together is an improper assembly tactic, since multiple arc pieces have to be put together simultaneously and multiple insertion directions exist.

Best manner of approaching the assembly issue is adding one piece at a time. This means, that after a single row has been fitted, adding another piece also requires only one insertion direction to be honored. However each succeeding element has to abide by two insertion directions and two arc connections to be made. Since the elements are generated in a manner that their extrusion direction is always normal to the initial surface, and the curvature is not rapidly changing along the surface, implementing a single puzzle piece between two adjacent pieces is possible. With increased curvature throughout the surface, the insertion direction vectors for two arc pieces can differ greatly. The problem in assembly process might occur with strong negatively curved surface geometry. In such cases the extrusion direction of the panels can be adjusted in the geometry design phase, so it accommodates for the problematic insertion direction. Furthermore, the Styrofoam elements, due to their material properties, can withstand slight bending and compression in order to fit into position and place. Similar work has been carried out regarding the insertion direction issue with freeform surface using rigid solid blocks [10].

Also, it is necessary to define an assembly strategy to reduce sanding or damaging the elements during insertion. The strategy is case specific. If the end goal is just to assemble the surface and leave it in one piece, the above described approach proves valid. In this research, the end goal was the ability to transport it easily and assemble and disassemble it accordingly. That is why the strategy is made to put together the structure by forming three large wall parts and connecting them with a stripe of maximum radius curvature puzzles. This differs from the main assembly strategy, but the end goal is different as well. As a consequence, the maximum radius curvature puzzles have to be joined between three adjacent pieces. However, the insertion process is made possible, given that those types of puzzles can be inserted along one direction respectively (Fig. 14).

Also, having three separate puzzle parts eases the transportation and handling process. Lastly, the finished structure needs to be sanded in order to smooth out any inconsistencies that the fabrication process could have left. This is mostly due to errors in manual reorienting of the cutting element, after the first cutting path is done. Minor rotations in both directions can cause loss of curvature continuity, inadequate assembly and/or reduction in size of the arc shapes and puzzle elements altogether. Due to the white color and vaguely noticeable seam pattern, several puzzle elements are removed from the finished curved wall in order to show the building block itself (Fig. 15).

Due to its form and shape, the wall is steady and sturdy on its own even with a few missing puzzle pieces.

8. Conclusion and future work

In this research we introduced a novel workflow of generating and fabricating freeform Styrofoam structures. The novel workflow consists of generating a unique code for the robot controller to follow, newly developed tools for tessellation procedures based on material properties and robotic fabrication workflow with regards to a previously designed freeform surface model. Hence, any freeform surface can be tessellated into either puzzle-like or box-like elements and prepared for the fabrication process. The Styrofoam box is cut in such a way that it leaves little scrap material, which can be reused for future purposes as well. Puzzle-like elements are an excellent way of tessellating curved surfaces and providing a solid bond between elements. The necessity for freeform structure comes from challenging issues in contemporary architecture, directed toward design of complex geometric shapes.

Future work will focus on improving the cutting tool in order to reduce human errors in the assembly process, as well as to increase possibilities in the design process. The Cutting tool can be modeled in such a way as to preserve the position and orientation of the cutting element once the cutting procedure is initialized, removing the need for the cutting element's reorientation. Therefore, the human error can be reduced to none and the finished elements are identical to their digital models. It makes the finished structure as close to the original design as possible.

Furthermore, the geometry generation phase can be upgraded by using a single extrusion vector for the panels, with increased



Fig. 15. The finished curved wall structure with missing puzzle elements to emphasize puzzle-like tessellation process

curvature or a saddle shaped surface patch. Given the difficulty during their assembly process, the locally defined arc insertion vectors can refer to a single row or a patch therefore allowing the panels to be fitted together. Generating a single extrusion vector for all panels can prove difficult with strong curvature surfaces and not cost-efficient for fabrication, due to unnecessarily thick bounding box for the elements and thin patches of panel surfaces. This approach can reduce the problematic assembly issues when dealing with local saddle shaped surfaces and their locally defined extrusion vector pertaining to surface normals.

Acknowledgments

This research was supported by the Serbian Ministry of Education and Science under contracts TR36042 and III44008; and by Provincial secretariat for science and technological development under contract 114-4512116/2011. The authors would also like to thank the company ABB for providing the licenses for the RobotStudio software.

Appendix A. Calculating quaternions from rotation matrix

According to van Waveren [41], the quaternions q_{xt}, q_{yt}, q_{zt} and q_{wt} are calculated from the rotation matrix T as follows:

- if $(t_{11} + t_{22} + t_{33}) > 0$

  $s = 2 \cdot \sqrt{t_{11} + t_{22} + t_{33} + 1}$

  $q^i_{wt} = 0.25 \cdot s$

  $q^i_{xt} = (t_{32} - t_{23}) / s$

  $q^i_{yt} = (t_{13} - t_{31}) / s$

  $q^i_{zt} = (t_{21} - t_{12}) / s$

- else if $(t_{11} > t_{22}$ and $t_{11} > t_{33})$

  $s = 2 \cdot \sqrt{t_{11} - t_{22} - t_{33} + 1}$

  $q^i_{wt} = (t_{32} - t_{23}) / s$

  $q^i_{xt} = 0.25 \cdot s$

  $q^i_{yt} = (t_{12} - t_{21}) / s$

  $q^i_{zt} = (t_{13} - t_{31}) / s$

- else if $(t_{22} > t_{33})$

  $s = 2 \cdot \sqrt{t_{22} - t_{11} - t_{33} + 1}$

  $q^i_{wt} = (t_{13} - t_{31}) / s$

  $q^i_{xt} = (t_{21} + t_{12}) / s$

  $q^i_{yt} = 0.25 \cdot s$

  $q^i_{zt} = (t_{32} + t_{23}) / s$

- else

  $s = 2 \cdot \sqrt{t_{33} - t_{22} - t_{11} + 1}$

  $q^i_{wt} = (t_{21} - t_{12}) / s$

  $q^i_{xt} = (t_{13} + t_{31}) / s$

  $q^i_{yt} = (t_{32} + t_{23}) / s$

  $q^i_{zt} = 0.25 \cdot s$

References

[1] S. Flöry, H. Pottmann, "Ruled surfaces for rationalization and design in architecture.", In, Proc. ACADIA (2010) 103–109.
[2] EvoluteTools, D.LOFT, http://www.evolute.at/software-en/evolutetools-for-rhino/evolutetools-d-loft.html. accessed: 2015-09-29,
[3] H. Pottmann, A. Schiftner, P. Bo, H. Schmiedhofer, W. Wang, N. Baldassini, J. Wallner, Freeform surfaces from single curved panels, ACM Trans. Graph 27 (2008) 76:1–76:10. http://dx.doi.org/10.1145/1360612.1360675.
[4] M. Eigensatz, M. Deuss, A. Schiftner, M. Kilian, N. Mitra, H. Pottmann, M. Pauly, Case Studies in Cost-Optimized Paneling of Architectural Freeform Surfaces, in: C. Ceccato, L. Hesselgren, M. Pauly, H. Pottmann, J. Wallner (Eds.), Advances in Architectural Geometry 2010, Springer, Vienna, 2010, pp. 4972. http://dx.doi.org/10.1007/978-3-7091-0309-8_4.
[5] M. Eigensatz, M. Kilian, A. Schiftner, N.J. Mitra, H. Pottmann, M. Pauly, Paneling architectural freeform surfaces, ACM Trans. Graph. 29 (2010) 1–10.
[6] Formtexx Ltd, http://www.formtexx.com, accessed: 2015-03-22.
[7] H. Pottmann, Architectural geometry as design knowledge, Archit. Des. 80 (2010) 7277. http://dx.doi.org/10.1002/ad.1109.
[8] F. Gramazio, M. Kohler, Digital Materiality in Architecture, Lars Müler Publishers Baden. 2008.
[9] N. Oxman, Towards a Material Ecology, Proceedings of the 32Nd Annual Conference of the Association for Computer Aided Design in Architecture, ACADIA, 2012, pp. 1920.
[10] P. Block, M. Rippmann, Digital Stereotomy: Voussoir Geometry for Freeform Masonry-Like Vaults Informed by Structural and Fabrication Constraints, IABSE-IASS, 2011. Symposium, London, United Kingdom, September 2011.
[11] Towering jigsaw puzzle curtain made of stone, http://www.mymodernmet.com/profiles/blogs/studio-gang-architects-marble-curtain. accessed: 2016-02-28,
[12] M. Rippmann, P. Block, New Design and Fabrication Methods for Freeform Stone Vaults Based on Ruled Surfaces, Computational Design Modelling, Springer. 2012, pp. 181189. http://dx.doi.org/10.1007/978-3-642-23435-4_21.
[13] Diamond-wire cutting workshop, http://www.robarch2012.org/workshops#Rotterdam accessed: 2015-03-22.
[14] R.J.B. Schipper, Manufacturing Double-Curved Elements Inprecast Concrete Using a Flexible Mould-First Experimental Results, Proceedings of the Fib Symposium, Concrete Engineering for Excellence and Efficiency, 2011, pp. 111.
[15] B. Kolarevic, Digital Production, in: B. Kolarevic (Ed.), Architecture in the Digital Age, Design and Manufacturing, Spoon Press, New York, 2003, pp. 46–88. http://dx.doi.org/10.1007/s00004-004-0025-4.
[16] C. Chih-Hsing, J.T. Chen, Tool path planning for five-axis flank milling with developable surface approximation, Int. J. Adv. Manuf. Technol. 29 (7-8) (2006) 707–713. http://dx.doi.org/10.1007/s00170-005-2564-6.
[17] W. Ping-Han, Y. Li, C. Chu, Optimized tool path generation based on dynamic programming for five-axis flank milling of rule surface, Int. J. Mach. Tools Manuf. Vol. 48 (11) (2008) 1224–1233. http://dx.doi.org/10.1016/j.ijmachtools.2008.03.009.
[18] Nedcam, http://www.nedcam.com, accessed: 2015-03-17.
[19] S. Keating, N. Oxman, Compound fabrication: a multifunctional robotic platform for digital design and fabrication, Robot. Comput. Integr. Manuf. 29 (2013) 439448. http://dx.doi.org/10.1016/j.rcim.2013.05.001.
[20] N. Oxman, J. Duro-Royo, S. Keating, B. Peters, E. Tsai, Towards robotic swarm printing, Archit. Des. 84 (2014) 108115. http://dx.doi.org/10.1002/ad.1764.
[21] T. Bonwetsch, F. Gramazio, M. Kohler, Digitally Fabricating Non-Standardised Brick Walls, Manubuild, 1St International Conference, Rotterdam, 2007,
[22] R. Bärtschi, M. Knauss, T. Bonwetsch, F. Gramazio, M. Kohler, Wiggled brick bond, Advances in Architectural Geometry 2010 (2010) 137147. http://dx.doi.org/10.1007/978-3-7091-0309-8_10.
[23] M. Raković, M. Jovanović, B. Borovac, B. Tepavčević, M. Nikolić, M. Papović, Design and Fabrication with Industrial Robot as Brick-Laying Tool and with Custom Script Utilization, Robotics in Alpe-Adria-Danube Region (RAAD), 2014 23Rd International Conference On, IEEE, 2014, pp. 15. http://dx.doi.org/10.1109/raad.2014.7002251.
[24] N. King, M. Bechthold, A. Kane, P. Michalatos, Robotic tile placement: tools, techniques and feasibility, Autom. Constr. 39 (2014) http://dx.doi.org/10.1016/j.autcon.2013.08.014.
[25] S. Reichert, T. Schwinn, R. La Magna, F. Waimer, J. Knippers, A. Menges, Fibrous structures: an integrative approach to design computation, simulation and fabrication for lightweight, glass and carbon fibre composite structures in architecture based on biomimetic design principles, 52 (2014) http://dx.doi.org/10.1016/j.cad.2014.02.005.
[26] G.Epps RoboFold, http://www.robofold.com/, accessed: 2015-03-16.
[27] S. Brell-Cokcan, J. Braumann, A New Parametric Design Tool for Robot Milling, Proceedings of the 30th Annual Conference of the Association for Computer Aided Design in Architecture, 2010, pp. 357363.
[28] E. Lloret, A.R. Shahab, M. Linus, R.J. Flatt, F. Gramazio, M. Kohler, S. Langenberg, Complex concrete structures: merging existing casting techniques with digital fabrication, Comput. Aided Des. 60 (2015) 4049. http://dx.doi.org/10.1016/j.cad.2014.02.011.
[29] J. Cortsen, J. Rytz, L.P. Ellekilde, D. sølvason, H.G. Petersen, Automated, Robot. Auton. Syst. 62 (2014) 13871397. http://dx.doi.org/10.1016/j.robot.2014.06.004.
[30] H. Brooks, D. Aitchison, A review of state-of-the-art large-sized foam cutting, rapid prototyping and manufacturing technologies, Rapid Prototyp. J. 16 (2010) 318327. http://dx.doi.org/10.1108/13552541011065713.
[31] Tailor Crete, http://gramaziokohler.arch.ethz.ch/web/e/forschung/164.html accessed 2016-02-28.
[32] D. Veenendaal, M. West, P. Block, History and overview of fabric formwork using fabrics for concrete casting, Struct. Concr. 12 (3) (September 2011) 164–177. http://dx.doi.org/10.1002/suco.201100014.
[33] M. Stavric, M. Kaftan, Robotic Fabrication of Modular Formwork for Non-Standard Concrete Structures, Digital physicality Proceedings of the 30th eCAADe conference, volume 2, 2012, pp. 12.
[34] Automated FoamDome, accessed: 2015-03-16, http://thibaultschwartz.com/.
[35] C. Wang, E. Gershon, Multi-dimensional dynamic programming in ruled surface fitting, Comput. Aided Des. 51 (2014) 39–49. http://dx.doi.org/10.1016/j.cad.2014.02.004.
[36] K. Tang, C. Wang, Modeling developable folds on a strip., J. Comput. Inf. Sci. Eng. 5 (1) (2005) 35–47. http://dx.doi.org/10.1115/1.1804206.
[37] C. Wang, K. Tang, Optimal boundary triangulations of an interpolating ruled surface, J. Comput. Inf. Sci. Eng. Vol. 5 (2005) 291–301. http://dx.doi.org/10.1115/1.2052850.
[38] L. Sass, Wood frame grammar, in: B. Martens, A. Brown (Eds.), Computer Aided Architectural Design Futures 2005, Springer, Netherlands, 2005, pp. 383–392. http://dx.doi.org/10.1007/1-4020-3698-1_36.
[39] A. Kilian, Fabrication of Partially Double-Curved Surfaces out of Flat Sheet Material through a 3D Puzzle Approach, ACADIA, volume 3, 2003,
[40] RAPID Reference Manual, System Data Types and Routines, ABB Robotics, 1-774.
[41] J. van Waveren, From Quaternion to Matrix and Back, Inc, Id Software, 2005.